

STAT GR5293

Search, Read, Cite: Building a Reliable Research Assistant Agent

Xiangquan Zeng (xz3457)

Jinxi Zhang (jz4015)

Inesh Vytheswaran (iav2109)

Instructor: Parijat Dube

May 2026

I. Introduction

1. Motivation

Large language models (LLMs) have become useful writing and reasoning assistants, but they remain limited in ways that directly affect research-style tasks. First, standard LLMs cannot access real-time information. Their knowledge is fixed at the time of training, which makes them unreliable for questions involving recent or changing information. Second, they can hallucinate by producing fluent but unsupported claims including citations that appear plausible but do not correspond to real sources. Third, even when an answer is factually correct, an ordinary LLM response often does not provide enough source level evidence for users to independently verify the claim.

This distinction between source retrieval and evidence verification is central to our project. A research assistant should not receive full credit simply for returning a URL or a plausible citation. In many real research workflows, the user needs an exact passage from the source that supports a specific claim. Therefore, we evaluate not only whether the system finds sources, but also whether it returns usable quotations, whether those quotations are verified as exact source text, and whether each major claim receives its own supporting evidence.

For a research agent, the goal is therefore not only to generate a fluent answer, but to generate an answer that is current, factually grounded, and traceable to specific sources. Tool-augmented LLMs address this problem by allowing models to call external tools such as search engines, databases, document parsers, and citation generators during inference. Instead of relying on parametric knowledge, the model can retrieve external evidence, inspect source material, and incorporate that information into a final response.

2. Research Questions

This project investigates how to build and evaluate a research assistant agent end-to-end, with particular attention to tool selection, factual grounding, and citation quality. We focus on two research questions:

RQ1. How does a ReAct-style agent loop compare to a single-pass chain-of-thought (CoT) approach in terms of tool selection accuracy, factual correctness, and efficiency on research-style questions?

RQ2. How does the number and diversity of available tools affect the agent’s task completion rate and reasoning behavior? In particular, can the agent learn to route specialized queries to specialized tools, and how does this interact with each strategy?

RQ3. How can we improve a verified quote-retrieval system to outperform a generic web-search baseline for citation-ready evidence retrieval? In particular, does the system return real quotes, verified quotes, and claim-level evidence rather than merely plausible URLs?

RQ4. Do the ReAct and CoT patterns observed on the curated benchmark generalize to a larger multi-hop question-answering benchmark such as HotpotQA? In particular, does ReAct’s iterative search loop improve answer accuracy on bridge and comparison questions, or does it mainly increase tool-use cost and latency?

3. Overview

To answer these questions, we built a LangChain-based research assistant agent using GPT-4o-mini as the backbone model. The agent has access to tools spanning web search, Wikipedia search, PDF fetcher, citation generator, verified quote-extraction, quote search (verified quotes from the loaded document), `summarize_active_document` (summarize the uploaded document), and finally a pipeline to find academic paper quotes. We implement two reasoning strategies: ReAct and single-pass chain-of-Thought. Using the same tool suite, we evaluate these strategies on HotpotQA, a multi-hop question-answering benchmark containing bridge and comparison questions. We measure exact match, token F1, number of tool steps, latency, and max-iteration failure rate. We additionally conduct a statistical significance test, an ablation study that disables the quote tool, and a focused evidence-retrieval benchmark comparing our verified quote pipeline against a generic web-search baseline. Separately, we evaluate our academic quote-finding pipeline against a generic web-search baseline using citation-readiness metrics, including real quote rate, verified quote rate, claim-level quote coverage, source quality, and exact-substring verification. The system is exposed through a Gradio demo that displays tool calls, outputs, and final answers.

II. Related Work

Our project builds on several research areas: tool-augmented language models, benchmarks for tool use, agent reasoning strategies, and retrieval-augmented generation with citation.

1. Tool-Augmented Language Models

Cobbe et al. (2021) introduced GSM8K and showed that fine tuning verifiers with calculator annotations improved math problem solving, making one of the earliest demonstrations that tool augmentation pays off during training. TALM (Parisi et al., 2022) generalized this idea by proposing a framework in which a language model emits tool-call tokens and consumes tool outputs, though it required thousands of human-annotated demonstrations per

task. PAL (Gao et al., 2022) reduced this annotation cost by having the LLM generate executable Python programs as intermediate steps, offloading numerical reasoning to an interpreter. Toolformer (Schick et al., 2023) went further by showing that an LLM can teach itself to use tools through self-supervised annotation: the model proposes API calls, executes them, and keeps only those whose outputs reduce next-token prediction loss. Modern function calling, as deployed in OpenAI and Anthropic APIs, formalizes this loop into a four-stage protocol. Our agent uses this protocol via LangChain’s tool-calling abstraction.

2. Benchmarks for Tool Use

Gorilla/APIBench (Patil et al., 2023) introduced retriever-augmented training over thousands of HuggingFace, PyTorch Hub, and TensorFlow Hub APIs, evaluated with AST sub-tree matching to detect hallucinated calls, but was limited to low API diversity and only single-turn dialogs. ToolAlpaca (Tang et al., 2023) addressed diversity by simulating 3000 multi-turn cases over 400 tools across 50 categories. ToolLLM/ToolBench (Qin et al., 2023) scaled further to 16464 real-world REST APIs with both single-tool and multi-tool dialogs. APIGen (Liu et al., 2024) built on ToolBench by adding a three-stage verification filter: execution checking, semantic checking, and quality verification to produce a smaller but higher-quality dataset.

These benchmarks share a limitation that motivates our work: they measure whether the correct API was called with the correct arguments, but do not evaluate whether the final synthesized answer is factually correct and properly cited. For a research assistant, the latter is the primary concern.

3. Agent Reasoning Strategies

Chain-of-Thought prompting (Wei et al., 2022; Lojima et al., 2022) showed that generating explicit intermediate reasoning steps improves performance on multi-step problems. ReAct (Yao et al., 2022) extended this to interactive settings by interleaving reasoning traces with actions: Thought => Action => Observation => Thought => . . . – allowing the model to revise its plan based on tool outputs. The original ReAct paper showed that this synergy outperformed either reasoning or acting alone on HotputQA and ALFWorld. Our project compares these two strategies on the same tool suite and backbone model, isolating the effect of the reasoning loop for research-style questions.

4. Retrieval-Augmented Generation and Citation Grounding

Retrieval-Augmented Generation (RAG) combines a retriever with a generator that conditions on retrieved passages, reducing hallucinations relative to closed-book generation.

However, attribution remains a hard problem: LLMs frequently paraphrase retrieved content in ways that drift from the source, or fabricate quotation marks around text that does not appear verbatim. QUILL (Xiao et al, 2025) framed this as the quotation generation problem and proposed enhancements for quote fidelity. Our quote-retrieval tool follows this spirit: rather than letting the LLM compose quotes, we retrieve candidate sentences from a parsed PDF, rerank them against the user’s claim, and verify that each returned quote is an exact substring of the source.

5. Positioning Our Contribution

Tool-use benchmarks measure API-call correctness but not end-to-end research quality. RAG systems evaluate retrieval and generation but assume a fixed retriever rather than an agent that decides when to retrieve. Agent papers introduce general frameworks but rarely run controlled comparisons on the same task suite with the same backbone. We address this gap by building an end-to-end research assistance that integrates retrieval, citation, and verification as first-class tools; comparing ReAct vs.CoT under matched conditions; and evaluating not only tool selection but also factual coverage, task completion, and the cost-quality tradeoff.

III. Methodology

1. System Architecture

The agent operates as a controller around a fixed LLM backbone (GPT-4o-mini). A user submits a research question, which is passed to the agent along with a registry of available tools and a strategy-specific system prompt. The agent emits structured tool calls (function name and JSON arguments), the runtime executes the corresponding Python function, and the result is returned as an observation. The agent either issues another tool call or produces a final answer. Every reasoning step, tool call, argument, and observation is logged for evaluation and for display in the Gradio demo.

Two design choices shape the system. First, all tools share the same LangChain @tool decorator, so the agent can be swapped between strategies without reengineering the tool layer. Second, the LLM backbone is held constant across both strategies, so any performance difference is attributable to the reasoning loop rather than the model.

2. Tool Suite

The agent has access to six tools, each targeting a distinct capability needed for research tasks:

1. **Web search** issues queries to the Serper API and returns the top organic results with title, snippet, and URL. The tool description directs the agent to use it for current facts, statistics, and recent events.
2. **Wikipedia search** queries the Wikipedia REST API for article summaries. Its narrow scope teaches the agent to prefer it for stable, well-established knowledge and avoid it for current events.
3. **PDF fetcher** downloads a PDF from a URL and extracts text using PyPDF2, returning a truncated representation with metadata. The agent calls this when it needs the full content of a paper or report referenced in earlier search results.
4. **Citation generator** takes structured source metadata (title, authors, year, URL, source type) and returns an APA-style citation string. It includes fallback logic that infers missing fields from URL patterns - for example, recognizing Wikipedia URLs or extracting publication years. Separating citation formatting into its own tool forces the agent to assemble references from retrieved metadata rather than generating them from memory.
5. **Quote search** retrieves verified, exact quotations from a loaded file. It dynamically parses the requested number of quotes from the user query and returns each candidate with character offsets, and a verification flag confirming the quote is an exact substring of the source. The active document can be inserted at runtime when a user uploads a PDF, TXT, or MD file through Gradio UI.
6. **Summarize active document** is a tool that can summarize or answer questions about the current loaded document. This allows the system to support uploaded PDF, TXT, or MD files rather than relying only on web search.

3. Agent Strategies

Both strategies share the same backbone, tool registry, and input format. They differ only in how they organize reasoning across tool calls. The ReAct agent uses LangChain's tool-calling agent abstraction and operates as an iterative loop. At each step it reasons about what information is still needed, selects a tool, observes the result, and decides whether to continue or produce a final answer. The loop terminates when the agent emits a final answer or reaches a step

cap of 15. The system prompt instructs it to be efficient, avoiding redundant searches and formatting citations inline rather than calling the citation tool repeatedly.

The Chain-of-Thought agent serves as a single-pass baseline. Given the question, it generates a JSON research plan listing all tool calls and their arguments upfront, executes those calls in sequence, and then synthesizes a final answer from their combined results. It has no opportunity to revise the plan based on intermediate observations. This makes it faster but less adaptive.

4. Refinement of Quote Finding Pipeline

In addition to the quote search, we implemented a longer academic quote retrieval pipeline called `find_paper_quotes` (FPQ). Since our goal was to optimize claim based quote retrieval, we used a snippet of a data science paper we wanted to find evidence for:

“XGBoost is a tree-based ensemble method that iteratively fits decision trees to minimize a specified loss function while controlling model complexity through regularization. The model is well suited for soil organic carbon prediction due to its ability to capture nonlinear relationships, feature interactions, and heterogeneous environmental responses across spatial and depth gradients. Find papers to reference and quotes to support this.”

Unlike the earlier `quote_search` tool, which retrieved passages from a single pre-loaded paper, this pipeline operates on an entire research paragraph. It first cleans the paragraph and decomposes it into atomic citation claims. This step is important because one paragraph often contains multiple claims requiring different sources.

For each claim, the system generates targeted search queries, retrieves candidate sources, and ranks them using source quality and claim relevance. Scholarly and primary sources are preferred over blogs or commercial explainers. The system then parses source text from PDFs or webpages, extracts candidate passages, and scores them using claim-term overlap, phrase coverage, keyword density, required-term coverage, structural cues, and source rank. Finally, it verifies that the selected quote appears as an exact substring of the parsed source text.

This refinement addresses a major weakness of generic web search baselines. A baseline can often return a plausible URL, but it may not provide an exact quote, and it cannot verify whether the quotation appears in the source. FPQ therefore evaluates evidence at the claim level: each citation-worthy claim should receive its own source, supporting quote, and verification flag.

5. Evaluation Framework

Our evaluation proceeds in two stages. First, we evaluate the full research assistant agent by comparing the ReAct and single-pass CoT strategies on a shared 10-question research benchmark. This stage measures whether each strategy selects the right tools, produces factually complete answers, completes the task successfully, and does so efficiently. Second, we evaluate the academic quote finding pipeline separately because citation-ready evidence requires stricter criteria than ordinary answer quality. In this stage, we compare our pipeline against a generic web-search baseline on 10 research-writing prompts and measure whether each system returns real quotes, verified exact-source quotes, claim-level evidence coverage, high-quality sources, and citation-ready outputs.

5.1 React vs CoT

We evaluate both agent strategies on a benchmark of 10 curated research questions spanning three question types: current-events lookups (favoring web search), definition and background questions (favoring Wikipedia), and quote extraction questions over the loaded source paper (favoring quote search). Each question is annotated with an expected tool set, a list of gold standard concept keywords, and a reference answer.

For each question and each agent we compute five metrics:

Tool Selection F1 – set-based F1 between expected and actual tools called, where precision penalizes unnecessary calls and recall penalizes missing required tools.

Keyword Coverage – the fraction of gold-standard concept keywords present in the final answer text, used as a reproducible proxy for factual correctness.

Answer Similarity – word-level F1 between the agent’s answer and the reference answer, measuring how well the response covers the expected content.

Task Completion Rate – the fraction of questions producing a non-empty, non-error final answer.

Reasoning Efficiency – the average number of agent steps and wall clock latency per question, quantifying the cost side of the quality-cost tradeoff.

For statistical testing we use a Mann-Whitney U test to determine whether observed differences between strategies are significant. To probe the effect of tool diversity, we re-run the

benchmark with `quote_search` removed from the tool-registry and compare full-tool versus ablated performance on both answer quality and reasoning efficiency.

5.2 Academic Quote Retrieval Pipeline

Because keyword coverage does not guarantee citation-ready output, we conduct a second evaluation focused on evidence retrieval. The benchmark contains 10 research-writing prompts, each with two citation-worthy claims. The goal is to retrieve one high-quality source and one supporting quotation for each claim.

We compare our academic quote finding pipeline with a generic web-search baseline. The pipeline decomposes each paragraph into atomic claims, searches for scholarly sources, parses webpages or PDFs, extracts and reranks candidate passages, and verifies that each selected quote appears as an exact substring of the source. The baseline performs a single web search using the full paragraph and asks the model to suggest sources and quotations from search snippets only; it does not open, parse, rerank, or verify sources.

We evaluate both systems using citation-oriented metrics. Quote Found measures whether a real quotation is returned, excluding placeholders such as “No exact quote available from snippet.” Verified Quote Found measures whether the quotation is confirmed as exact source text. Quote Density and Verified Quote Density measure real and verified quotes per expected claim, capped at 1.0. Complete Claim Quote Coverage requires every claim to receive a supporting quote, while Complete Verified Claim Quote Coverage requires every claim to receive a verified quote.

We also assess source quality and overall citation readiness. Mean Source Quality rewards scholarly or primary sources such as arXiv, Nature, Frontiers, ACM, IEEE, and PNAS, while penalizing weaker sources such as blogs, Medium posts, and commercial explainers. Citation Readiness combines source presence, quotation availability, verification, claim coverage, source quality, and quote completeness into a single score from 0 to 1. Together, these metrics test whether a system produces evidence that is not only relevant-looking, but usable for research writing.

5.3 HotpotQA Multi-Hop Evaluation

To test whether the ReAct and CoT patterns generalize beyond the curated benchmark, we also evaluate both agents on 100 questions sampled from the HotpotQA validation set. HotpotQA is a multi-hop QA benchmark with bridge questions, which require finding an intermediate entity, and comparison questions, which require comparing two entities. Our sample contains 85 bridge questions and 15 comparison questions.

Both agents use the same GPT-4o-mini backbone and tool registry. ReAct chooses tools iteratively after observing results, while CoT generates a fixed tool-use plan before execution. To make the comparison fair, CoT is required to use at least one search tool for factual questions.

Both systems return answers in the JSON format `{"answer": "short final answer"}` so that evaluation is based only on the parsed answer rather than explanations or citations. We measure exact match, token F1, average tool steps, latency, and max-iteration failure rate. For significance testing, we use McNemar’s exact test for exact match and Wilcoxon signed-rank tests for token F1, tool steps, and latency. We also report results separately for bridge and comparison questions.

IV. Experimental Setup

All experiments used GPT-4o-mini as the backbone language model through the OpenAI API with temperature set to 0 to reduce output variability. The ReAct agent was run with a maximum step cap of 8, while the single-pass CoT baseline generated its full tool-use plan before execution and therefore did not use an iterative step cap. Both strategies used the same tool registry, prompts, benchmark inputs, and logging format to ensure that differences in performance were attributable to the reasoning strategy rather than differences in system configuration.

The system was implemented in Google Colab on a standard CPU runtime. The main dependencies were LangChain, the OpenAI Python SDK, wikipedia-api, PyPDF2, sentence-transformers, pandas, scipy, and Gradio. In addition, we included OCR-related dependencies, including pdf2image, pytesseract, Poppler, and Tesseract, to support text extraction from scanned or image-based PDFs. In the main pipeline, selectable-text PDFs were processed with PyPDF2, while OCR tools provide a fallback option when direct text extraction is not sufficient. Web search was performed using the Serper API. API keys were stored in Colab Secrets rather than hardcoded in the notebook.

All runs included a retry wrapper with exponential backoff to handle transient API or network errors. We recorded latency, tool outputs, parsed quotations, source metadata, and final responses for each benchmark item. No model fine-tuning was performed; all components were used in inference mode.

V. Results and Analysis

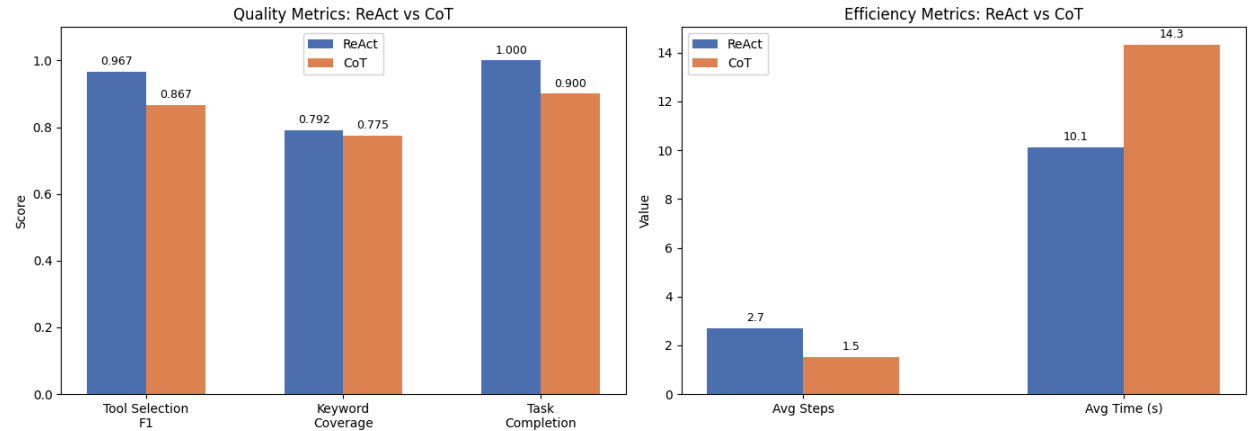
1. ReAct vs. CoT: Overall Comparison

Table 1 summarizes the performance of both agent strategies across all 10 benchmark questions.

Metric	ReAct	CoT
Avg Tool Selection F1	0.967	0.867
Avg Keyword Coverage	0.792	0.775
Avg Steps	2.7	1.5
Avg Time (seconds)	10.1	14.3
Task Completion Rate	1.000	0.900

ReAct achieved a 100% task completion rate, while CoT reached 90% – every ReAct question produced a valid answer, but CoT failed on q09, where its single-pass planner routed to FPQ instead of quote_search and produced no usable response. Keyword coverage favored ReAct (0.792 vs. 0.775), indicating that the iterative reasoning loop produces more factually complete answers. The difference is driven by questions like q01 and q06 where ReAct issued follow-up searches after initial results were insufficient, while CoT committed to its upfront plan and missed key concepts. On q06 (“current state of quantum computing”), CoT scored 0.00 keyword coverage with a single web_search call, while ReAct recovered to 1.00 by issuing additional searches.

Tool selection F1 also favored ReAct (**0.967 vs. 0.867**). The difference was small because both agents usually selected the expected tools, but ReAct achieved perfect tool routing on most questions and recovered better on multi-source questions such as q03. CoT remained competitive because it used fewer calls and often selected the core expected tool directly, but its single-pass planner misrouted q09 to a similar but incorrect specialized tool, dragging down its average



Surprisingly, ReAct was also faster on average. ReAct averaged 2.7 steps and 10.1 seconds per question, compared to 1.5 steps and 14.3 seconds for CoT. CoT retains an advantage

in tool call count and API cost, but its synthesis step was unusually slow on two outlier questions (q08 at 38.4 and q10 at 37.3s), erasing its expected wall-clock lead at this benchmark size.

2. Statistical Significance

A Mann-Whitney U test found no statistically significant difference between the two strategies on either keyword coverage ($U = 50.500$, $p = 1.000$) or tool selection F1 ($U = 55.500$, $p = 0.543$). This indicates that while ReAct trends toward higher factual coverage, the difference is not statistically distinguishable from noise given our benchmark size of 10 questions. A larger benchmark would be needed to confirm whether the observed advantage is reliable.

3. Tool Routing Behavior

Examining individual questions reveals that the two strategies differ most on specialized tasks. For quote-extraction questions (q04, q09), ReAct correctly routed to `quote_search` in both cases. CoT Routed correctly on q04 but on q09 chose FPQ instead of `quote_search`, resulting in tool F1 = 0.0 and an empty answer for that question. This suggests that ReAct’s iterative loop is more reliable at distinguishing between similar specialized tools (quote extraction from a loaded document vs. quote retrieval across external papers), while CoT’s single-pass planner can confuse adjacent specialized tools.

For factual questions (q01, q02, q05, q10), both strategies performed similarly by selecting `web_search` and `wikiedia_search` as expected. The strategies converge when the task is straightforward and diverge when the task requires tool-specific routing.

4. Ablation Study: Effect of Quote Search Tool

To test the effect of tool diversity (RQ2), we re-ran the quote-extraction questions with `quote_search` removed from the tool registry.

Question	Setting	Tool called	# Tool calls	Keyword Coverage
q04	With <code>quote_search</code>	<code>quote_search</code>	1	1.00
	Without <code>quote_search</code>	<code>web_search</code> , <code>fetch_pdf</code> , <code>web_search</code> , <code>fetch_pdf</code>	4	1.00
q09	With <code>quote_search</code>	<code>quote_search</code>	1	0.33

Without quote_search	Web_search, fetch_pdf, fetch_pdf	3	0.33
----------------------	-------------------------------------	---	------

Without quote_search, the agent fell back to web_search and fetch_pdf, requiring 4-5 tool calls instead of 1 to attempt the same task. Keyword coverage remained similar in both conditions, suggesting the agent can find relevant information through alternative tools. However, the ablated agent cannot provide verified, exact-substring quotes with character locations – it can only paraphrase or summarize what it finds online. This shows that quote_search does not improve keyword-level metrics but provides a qualitatively different output (verified quotes with provenance) that general-purpose tools cannot replicate.

5. Error Analysis

We observed three recurring failure modes across both strategies. First, the citation generator occasionally received hallucinated author names – for example, the agent extracted “Shardlow, M.” from a search snippet when the correct name was “Shazeer, N.” This occurs because the agent infers metadata from noisy search result text rather than from structured source data. Second, fetch_pdf failed on URLs that were not direct PDF links (blog posts, HTML pages), though the agent recovered gracefully by trying alternative URLs or falling back to web search. Third, quote_search returned quotes with negative reranker scores on off-topic queries (e.g., searching for “GPT-4 architecture” in the Transformer paper), indicating the tool was called inappropriately. The updated tool description limiting it to the Transformer paper reduced but did not eliminate this issue.

6. Evidence Retrieval Benchmark

The ReAct versus CoT evaluation measures tool selection and answer coverage, but those metrics do not fully capture citation quality. To evaluate citation readiness directly, we ran a separate 10-example evidence-retrieval benchmark comparing FPQ against a generic web-search baseline.

The results show a clear distinction between retrieving plausible sources and retrieving verified evidence. Both systems found at least one source for every example, so source found rate was 1.00 for both. However, the baseline often returned weak sources, incomplete snippets, or placeholders such as “No exact quote available from snippet.” After filtering out such placeholders, the baseline returned a real quote on only 50% of examples, while FPQ returned real quotes on 100% of examples.

The largest difference appears in verification. The baseline had a verified quote found rate of 0.00 because it does not open documents or check whether quotation text appears in the

source. In contrast, FPQ achieved a verified quote found rate of 1.00 and a complete verified claim quote coverage rate of 1.00. This means that every claim in every benchmark example received at least one verified supporting quote.

The source quality metric also favored FPQ. The verified pipeline achieved a mean source quality of 1.000, while the baseline achieved 0.363. This reflects a qualitative pattern in the outputs: FPQ tended to retrieve scholarly or primary sources such as arXiv papers, Nature articles, and Frontiers articles, whereas the baseline often returned Medium posts, blogs, commercial explainers, or search-result snippets.

Importantly, the baseline remained competitive on surface-level metrics. Its mean keyword coverage was 0.930 and its expected claim coverage was 0.950, close to FPQ at 0.980 and 1.000 respectively. This shows why keyword-based evaluation alone is insufficient for research agents. A system can mention the right concepts while still failing to provide usable evidence. The citation readiness score captures this gap: FPQ achieved a mean citation readiness of 0.990, compared to 0.416 for the baseline.

The tradeoff is latency. The verified quote pipeline averaged 19.934 seconds per example, compared to 3.509 seconds for the generic baseline. This reflects the additional work required to decompose claims, retrieve sources, parse documents, extract quotes, rerank evidence, and verify exact substrings. For casual lookup tasks, the faster baseline may be acceptable. For research writing, where exact support and provenance matter, the added latency buys substantially higher evidence quality.

7. Academic Quote Benchmarking

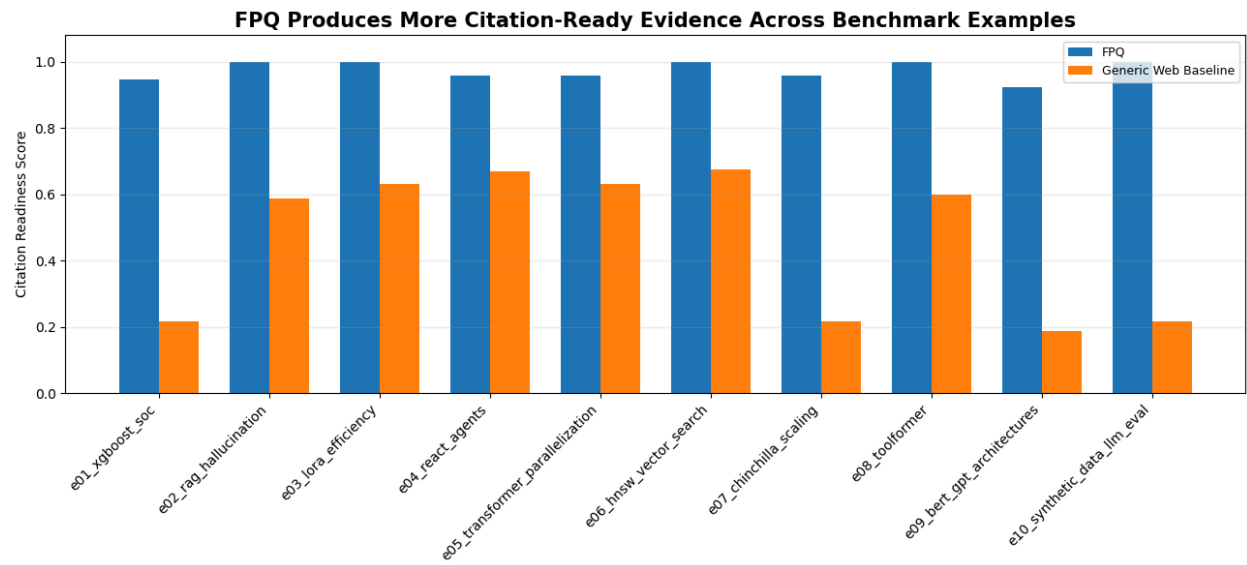
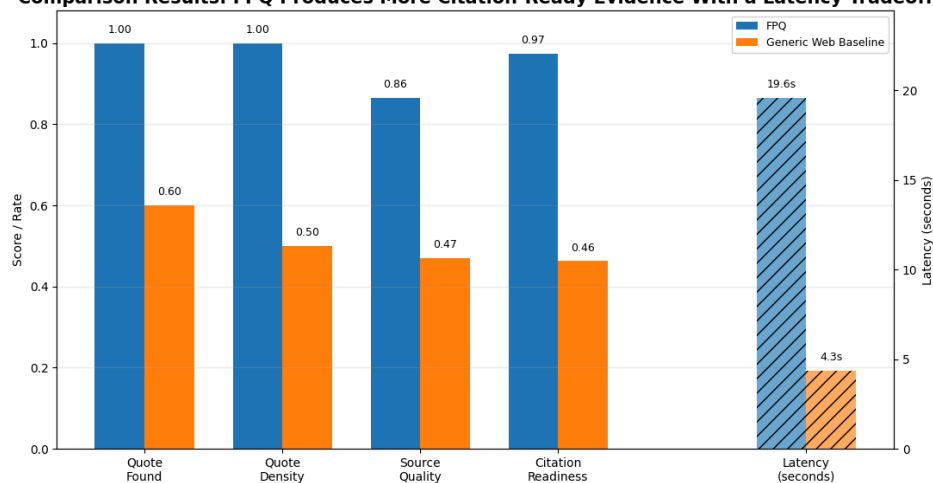


Table 3 reports the results of the evidence-retrieval benchmark comparing FPQ) with the generic web-search baseline:

Metric	FPQ	Generic Web Baseline
Mean latency (sec)	14.580	5.562
Source found rate	1.000	1.000
Quote found rate	1.000	0.600
Complete claim quote coverage	1.000	0.200
Mean quote density	1.000	0.400
Mean quote completeness	1.000	0.333
Mean source quality	1.000	0.363
Mean keyword coverage	0.888	0.967
Mean expected claim coverage	1.000	0.967
Mean citation readiness	0.983	0.4138
Mean number of URLs	2.200	2.300
Mean number of quotes	2.000	1.100
Failure rate	0.000	0.000

Comparison Results: FPQ Produces More Citation-Ready Evidence With a Latency Tradeoff



8. HotpotQA Multi-Hop QA Results

To test whether our curated-benchmark findings generalized to multi-hop QA, we evaluated ReAct and CoT on 100 HotpotQA validation questions: 85 bridge questions and 15 comparison questions.

Overall, CoT slightly outperformed ReAct on answer accuracy, but the difference was not statistically significant. ReAct achieved 0.370 exact match and 0.469 token F1, while CoT achieved 0.390 exact match and 0.487 token F1. McNemar’s exact test for exact match gave $p = 0.8145$, and the Wilcoxon signed-rank test for token F1 gave $p = 0.6788$. Therefore, we cannot conclude that either strategy was more accurate overall.

However, ReAct was significantly less efficient. It averaged 4.44 tool steps and 8.75 seconds per question, compared with 1.23 tool steps and 2.40 seconds for CoT. ReAct also hit the maximum-iteration limit on 27% of questions, while CoT had no max-iteration failures. The Wilcoxon tests for both tool steps and latency were significant with $p < 0.001$.

Table 4. Overall HotpotQA Results

Metric	ReAct	CoT
Exact Match	0.370	0.390
Token F1	4.440	1.230
Average Tool Steps	0.469	0.487
Average Time (sec)	8.751	2.399
Max-Iteration Failure Rate	0.270	0.000

VI. Discussion

1. CoT Is Surprisingly Competitive

The most notable finding is that the single-pass CoT agent matched ReAct on most metrics despite having no ability to revise its plan. On tool selection F1, ReAct scored higher (0.967 vs 0.867) because CoT remains competitive because it uses fewer tool calls per question and lower API cost, but it no longer achieves a wall-clock advantage at this benchmark size.

2. HotpotQA Confirms the Cost of ReAct

The HotpotQA evaluation strengthens the efficiency finding from the curated benchmark. On the 100-question sample, ReAct did not produce a statistically significant accuracy gain over CoT. CoT slightly outperformed ReAct on exact match and token F1, while ReAct required substantially more tool calls and longer runtime. This suggests that ReAct’s iterative search loop is not automatically beneficial for multi-hop QA. The agent can keep searching after each observation, but without strong stopping rules this adaptivity can turn into over-searching.

The 27% max-iteration failure rate is especially important. These failures show that ReAct’s main weakness was not lack of information access, but control: the agent sometimes failed to decide when it had enough evidence to answer. In contrast, CoT’s fixed plan limited unnecessary exploration, making it faster and more reliable even though it was less adaptive.

The question type breakdown gives a more nuanced picture. On bridge questions, ReAct performed slightly better than CoT, but the difference was not statistically significant. This fits the intuition that ReAct may help when a question requires sequential lookup: the agent can search for an intermediate entity, observe the result, and then search again.

However, on comparison questions, CoT performed better, and its token F1 advantage was statistically significant. Comparison questions often have a simpler structure: retrieve information about two entities, compare them, and answer. A fixed plan is well suited to this pattern, while ReAct may waste steps exploring extra sources or reformulating searches.

3. ReAct Is Adaptive but Requires Better Control

Where the two strategies diverged most was specialized tool routing (§V.3). CoT’s single-pass planner defaults to familiar, general-purpose tools and does not explore the full tool registry. This is consistent with a known limitation of single-pass planning: the model must anticipate the correct tool sequence before seeing any evidence, which is difficult for specialized tools the model has less experience with. For systems with a diverse tool suite, ReAct’s iterative exploration is a meaningful advantage.

The combined HotpotQA results suggest that ReAct’s value is not simply “more reasoning is better.” ReAct is useful when the task requires adaptive tool routing, such as deciding whether to use `quote_search`, `web_search`, or `fetch_pdf`. However, HotpotQA shows that adaptivity also introduces a control problem. The agent needs better stopping criteria, query reformulation, and confidence estimation; otherwise, it can spend additional tool calls without improving the answer. Future versions of the system should include a stopping rule that checks whether the current evidence is sufficient before issuing another search.

4. Tool Diversity Matters, But Not How We Expected

As the ablation showed (§V.4), removing `quote_search` does not substantially reduce keyword coverage. However, the evidence-retrieval benchmark shows why this conclusion is incomplete. Keyword-level metrics can make a generic system look competitive even when it fails to produce verified evidence. The generic web baseline achieved 0.930 keyword coverage and 0.950 expected claim coverage, but its verified quote density was 0.000 and its complete verified claim coverage was 0.000. In other words, it often discussed the right concepts but did not provide source-verifiable quotations.

This distinction matters for research assistance. A student or researcher does not only need a plausible source; they need evidence that can be inspected, quoted, and cited. The `quote_search` and FPQ pipelines provide this by verifying exact source text and returning claim-level evidence. Thus, specialized tools may not improve shallow factual coverage metrics, but they can dramatically improve provenance, trustworthiness, and citation readiness.

The HotpotQA results also show that tool diversity alone is not enough. ReAct had access to the same tools as CoT and used them more often, but this did not translate into higher accuracy. The value of a tool suite depends not only on having the right tools available, but also on whether the agent can decide when to use them and when to stop.

5. FPQ: Verified Evidence Requires Different Metrics

The evidence-retrieval benchmark shows that citation-ready research assistance requires stricter metrics than ordinary answer quality. Both FPQ and the generic web-search baseline found at least one source for every example, but source retrieval alone did not guarantee usable evidence. FPQ returned real quotes for all examples, while the baseline returned real quotes for only 60% after excluding placeholders such as “No exact quote available from snippet.” PQ also produced higher-quality sources, with a mean source quality of 0.932 compared with 0.420 for the baseline.

Importantly, the baseline remained competitive on surface-level metrics: its keyword coverage was 0.967 and expected claim coverage was 0.967, close to FPQ’s 0.888 and 0.980. This shows why keyword-based evaluation can overstate research quality. A system may mention the right concepts while still failing to provide quotations that can be inspected, verified, and cited. Citation readiness captured this difference more clearly: FPQ scored 0.983 compared with 0.438 for the baseline.

The tradeoff is latency. FPQ averaged 14.58 seconds per example, compared with 5.56seconds for the baseline, because it performs claim decomposition, targeted search, document parsing, passage extraction, reranking, and exact-substring verification. For casual

lookup tasks, the faster baseline may be acceptable. For research writing, where provenance and quote fidelity matter, FPQ’s additional latency is justified by its stronger citation readiness.

Ultimately, we found FPQ to be highly successful on the original XGBoost and soil organic carbon query. For the first claim, FPQ retrieved the landmark XGBoost paper, *XGBoost: A Scalable Tree Boosting System* (Chen & Guestrin, 2016), showing that the pipeline could identify canonical sources for well-known methods. The selected quote directly supported the claim that XGBoost minimizes a regularized objective with both a loss term and a model-complexity penalty. For the second claim, which involved a much more specialized topic, FPQ still retrieved a strong domain-specific source. In particular, it found the quote: “XGBoost and Gradient Boosting Machines have consistently yielded strong predictive performance in digital soil mapping and carbon stock assessments due to their ensemble learning structure and ability to manage missing or noisy data” (Ghorbal et al., 2025). This result shows that FPQ can handle both canonical machine learning and narrower applied science claims, while returning source grounded evidence rather than only plausible URLs.

6. Limitations

Several limitations constrain our findings. First, the curated benchmark contains only 10 questions, but the HotpotQA experiment expanded the evaluation to 100 multi-hop questions. We could have scaled to the full HotpotQA validation set, but were limited by Serper API query costs and runtime. With cached search results or a larger query budget, the same code could run on the full dataset. Second, `quote_search` operates on a single active document at a time. The Gradio UI lets the user upload and swap the active document, but the agent itself does not autonomously select papers based on the user’s query. A production system would automatically retrieve and load relevant papers per question, which introduces additional retrieval latency and parsing challenges we did not address. Third, our keyword coverage and answer similarity metrics are surface-level proxies for factual correctness. Replacing them with embedding-based similarity (e.g., BERTScore) or LLM-as-a-judge evaluation would provide a more meaningful accuracy signal. Fourth, the quote retrieval pipeline currently uses `infloat/e5-small-v2` for sentence embedding; upgrading to larger models such as `e5-large-v2` would likely improve retrieval quality. Finally, the system was tested with a single backbone model (GPT-4o-mini); comparing against an open-source alternative like Llama 3 or Qwen would test whether the findings generalize across model families.

VII. Conclusion

We built and evaluated a tool-augmented research assistant agent that integrates web search, encyclopedic lookup, document parsing, citation formatting, and verified quote extraction into a unified system. By comparing ReAct and chain-of-Thought strategies under matched conditions, we found that both achieve comparable answer quality, but with different strengths: ReAct produces more comprehensive answers and routes effectively to specialized tools, while CoT delivers similar results at roughly half the latency and cost. Neither strategy’s advantage was statistically significant at our benchmark size, suggesting that strategy selection in practice should be guided by the use case: ReAct for complex, multi-tool queries and CoT for straightforward lookups where speed matters.

The quote retrieval pipeline, developed through three iterations from keyword matching to semantic retrieval to cross-encoder reranking, demonstrates that verified citation grounding can be integrated as a first-class tool within an agent framework. The ablation confirmed that while general-purpose tools can match keyword-level metrics, they cannot replicate the provenance guarantees that a dedicated quote tool provides.

Several directions remain for future work: expanding the benchmark to strengthen statistical power, supporting paper loading so the quote tool generalizes beyond a single pre-loaded document, replacing surface-level metrics with embedding-based or LLM-as-judge evaluation, and testing whether these findings hold across different backbone models. The system, evaluation code, and benchmark are publicly available in our GitHub repository.

References:

1. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). *Training verifiers to solve math word problems*. arXiv. <https://arxiv.org/abs/2110.14168>
2. Parisi, A., Zhao, Y., & Fiedel, N. (2022). *TALM: Tool augmented language models*. arXiv. <https://arxiv.org/abs/2205.12255>
3. Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., & Neubig, G. (2022). *PAL: Program-aided language models*. arXiv. <https://arxiv.org/abs/2211.10435>
4. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). *Toolformer: Language models can teach themselves to use tools*. arXiv. <https://arxiv.org/abs/2302.04761>
5. Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2023). *Gorilla: Large language model connected with massive APIs*. arXiv. <https://arxiv.org/abs/2305.15334>
6. Tang, Q., Deng, Z., Lin, H., Han, X., Liang, Q., Cao, B., & Sun, L. (2023). *ToolAlpaca: Generalized tool learning for language models with 3000 simulated cases*. arXiv. <https://arxiv.org/abs/2306.05301>
7. Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., & Sun, M. (2023). *ToolLLM: Facilitating large language models to master 16000+ real-world APIs*. arXiv. <https://arxiv.org/abs/2307.16789>
8. Liu, Z., Hoang, T., Zhang, J., Zhu, M., Lan, T., Kokane, S., Tan, J., Yao, W., Liu, Z., Feng, Y., Murthy, R., Yang, L., Savarese, S., Niebles, J. C., Wang, H., Heinecke, S., & Xiong, C. (2024). *APIGen: Automated pipeline for generating verifiable and diverse function-calling datasets*. arXiv. <https://arxiv.org/abs/2406.18518>
9. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). *Chain-of-thought prompting elicits reasoning in large language models*. Advances in Neural Information Processing Systems, 35. <https://arxiv.org/abs/2201.11903>
10. Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). *Large language models are zero-shot reasoners*. Advances in Neural Information Processing Systems, 35. <https://arxiv.org/abs/2205.11916>
11. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing reasoning and acting in language models*. arXiv. <https://arxiv.org/abs/2210.03629>
12. Xiao, J., Zhang, B., He, Q., Liang, J., Wei, F., Chen, J., Liang, Z., Yang, D., & Xiao, Y. (2024). *QUILL: Quotation generation enhancement of large language models*. arXiv. <https://arxiv.org/abs/2411.03675>
13. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems, 30. <https://arxiv.org/abs/1706.03762>

14. Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. <https://arxiv.org/abs/1603.02754>
15. Ben Ghorbal, A., Grine, A., Eid, M. M., & El-kenawy, E.-S. M. (2025). Sustainable soil organic carbon prediction using machine learning and the ninja optimization algorithm. *Frontiers in Environmental Science*, 13, 1630762. <https://doi.org/10.3389/fenvs.2025.1630762>